

RMSNorm#

<https://pytorch.org/docs/stable/generated/torch.nn.RMSNorm.html>

Description:

Applies Root Mean Square Layer Normalization over a mini-batch of inputs. This layer implements the operation as described in the paper [Root Mean Square Layer Normalization](#). The RMS is taken over the last D dimensions, where D is the dimension of `normalized_shape`. For example, if `normalized_shape` is (3, 5) (a 2-dimensional shape), the RMS is computed over the last 2 dimensions of the input. `normalized_shape` (int or list or `torch.Size`) – input shape from an expected input of size

Function Signature

```
class torch.nn.RMSNorm(normalized_shape, eps=None,
                        elementwise_affine=True, device=None, dtype=None)[source]#
```

Parameters

Shape:

```
extra_repr()[source]#
```

Return type

Returns:

Tensor

Code Examples

```
>>> rms_norm = nn.RMSNorm([2, 3])
>>> input = torch.randn(2, 2, 3)
>>> rms_norm(input)
```

Detailed Documentation

Documentation

Applies Root Mean Square Layer Normalization over a mini-batch of inputs.

This layer implements the operation as described in the paper Root Mean Square Layer Normalization

The RMS is taken over the last D dimensions, where D is the dimension of `normalized_shape`. For example, if `normalized_shape` is (3, 5) (a 2-dimensional shape), the RMS is computed over the last 2 dimensions of the input.

`normalized_shape` (int or list or torch.Size) –

input shape from an expected input of size

If a single integer is used, it is treated as a singleton list, and this module will normalize over the last dimension which is expected to be of that specific size.

`eps` (Optional[float]) – a value added to the denominator for numerical stability. Default: `torch.finfo(x.dtype).eps`

`elementwise_affine` (bool) – a boolean value that when set to `True`, this module has learnable per-element affine parameters initialized to ones (for weights). Default: `True`.

Input: $(N,*) (N, *) (N,*)$

Output: $(N,*) (N, *) (N,*)$ (same shape as input)

Return the extra representation of the module.

Runs the forward pass.

Resets parameters based on their initialization used in `__init__`.